Nam Tu - 153076622 - Graph Analysis

Task 4.1

There is not much difference between the final vector containing the probabilities of each node after multiple iterations (1000, for example, in this work). It can be seen that nodes with more arrows pointing to it will have higher probability mass (for example, node c has the most arrows so it has the highest value)

To look into more details, a fraction of the probability of going to node c was split between node a and b with taxation implemented.

Taxation is primarily designed to counter dead ends which isn't really a thing in this diagram, so this is why it doesn't shine much.

Task 4.1a - No taxation

In [ ]:
```
import numpy as np

# Create inital matrix of "movements"
M = np.array([
    [1/3, 1/2, 0],
    [1/3, 0, 1/2],
    [1/3, 1/2, 1/2]
])

# Initial probabilities
x = np.array([
    [1/3],
    [1/3],
    [1/3]
])

#Applying matrix M to x
for i in range(1000):
    x = M.dot(x)

print("PageRank with no taxation:")
print(x)
```

```
PageRank with no taxation:
[[0.23076923]
 [0.30769231]
 [0.46153846]]
```

Task 4.1b - 0.8 taxation

In [ ]:
```
# Create inital matrix of "movements"
M = np.array([
    [1/3, 1/2, 0],
    [1/3, 0, 1/2],
```

```
    [1/3, 1/2, 1/2]
])

# Initial probabilities
x = np.array([
    [1/3],
    [1/3],
    [1/3]
])

b = 0.8
#Applying matrix M to x
for i in range(1000):
    x = (b*M).dot(x) + (1-b)*np.ones((3,1))/3

print("PageRank with 0.8 taxation:")
print(x)
```

```
PageRank with 0.8 taxation:
[[0.25925926]
 [0.30864198]
 [0.43209877]]
```

Run this in pyspark

Important pyspark initialization routines

In [1]:
```
import pyspark
from pyspark.sql import *
try:
    sc = pyspark.SparkContext('local[*]',environment = {})
except:
    sc = sc
#create sqlcontext on the spark, enables the use of the SQL queries below
#sqlContext = SQLContext(sc)

sqlContext = pyspark.sql.SparkSession(sc)
```

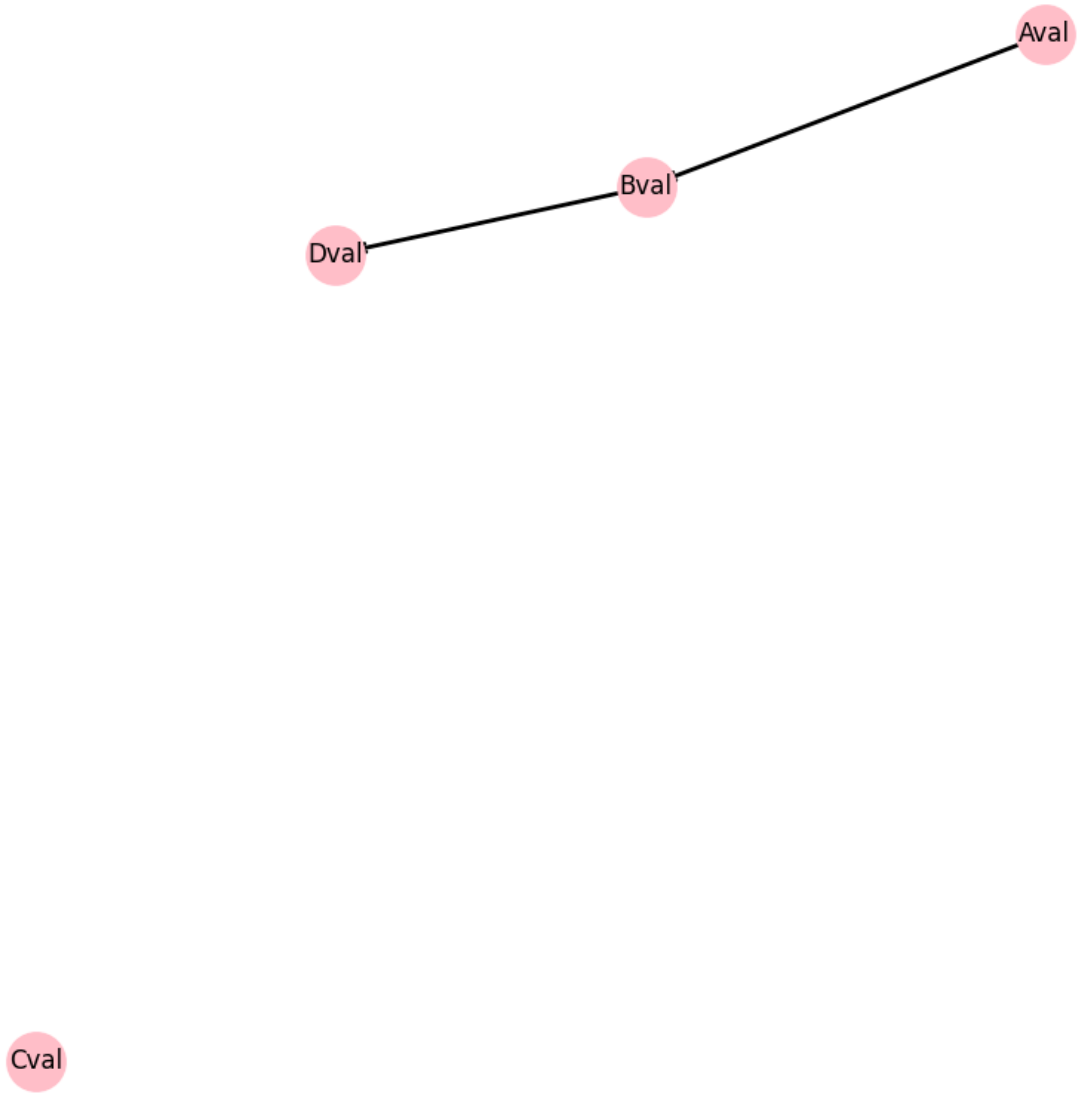
Do not mind the warning about deprecation

Below is a tiny example on how to create a graph

In [5]:
```
!pip install networkx
import networkx as nx
import matplotlib.pyplot as plt
# Create a GraphFrame
G = nx.Graph()
#Add nodes (vertices)
G.add_node("a",name="Aval")
G.add_node("b",name="Bval")
G.add_node("c",name="Cval")
G.add_node("d",name="Dval")
#make weighted connections (edges)
G.add_weighted_edges_from([("a", "b", 0.25), ("b", "d", 0.75)])

#show the contents of the graph nodes and edges
```

```python
print(G.nodes)
print(G.edges)
#initialize layout for figure
pos = nx.spring_layout(G, seed=42)
#get labels from graph
labes=[]
list_nodes=list(G.nodes)
for i in range(0,len(G)):
    au=list_nodes[i]
    bu=G.nodes[au]['name']
    labes.append(au)
    labes.append(bu)
print(labes)
labels = {labes[i]: labes[i + 1] for i in range(0, len(labes), 2)}
#print graph
plt.figure(figsize=(10, 10))
labels = nx.draw_networkx_labels(G, pos,labels=labels)
print(labels)
#nodes and edges variables are not needed, it is  result from plotting
nodes = nx.draw_networkx_nodes(G, pos,node_size=800,node_color='pink')
edges = nx.draw_networkx_edges(
    G,
    pos,
    arrows=True,
   arrowstyle="->",
    arrowsize=10,
    width=2,
)
plt.axis("off")
plt.show()
```

```
Requirement already satisfied: networkx in C:\Users\haina\AppData\Local\Programs
\Python\Python311\Lib\site-packages (3.6.1)
['a', 'b', 'c', 'd']
[('a', 'b'), ('b', 'd')]
['a', 'Aval', 'b', 'Bval', 'c', 'Cval', 'd', 'Dval']
{'a': Text(0.30389938970763614, 0.5166412507223705, 'Aval'), 'b': Text(0.07479419
616928383, 0.29120297601273976, 'Bval'), 'c': Text(-0.2752433885491036, -1.0, 'Cv
al'), 'd': Text(-0.10345019732781642, 0.19215577326488975, 'Dval')}
```



Task 4.2.1 (3 p)

1. Modify the code below using the above example to form the graph that is shown in the Mining Massive Datasets book in figure 5.1 (figure below). Additionally, give random numbers to the edges (add random numbers in g.es["weights"]).

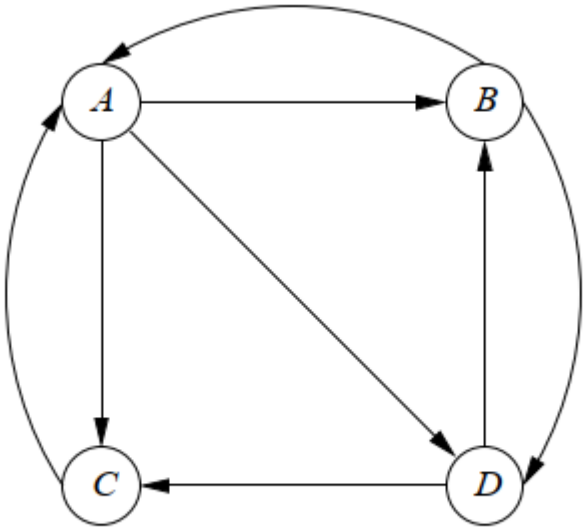


Figure 5.1: A hypothetical example of the Web

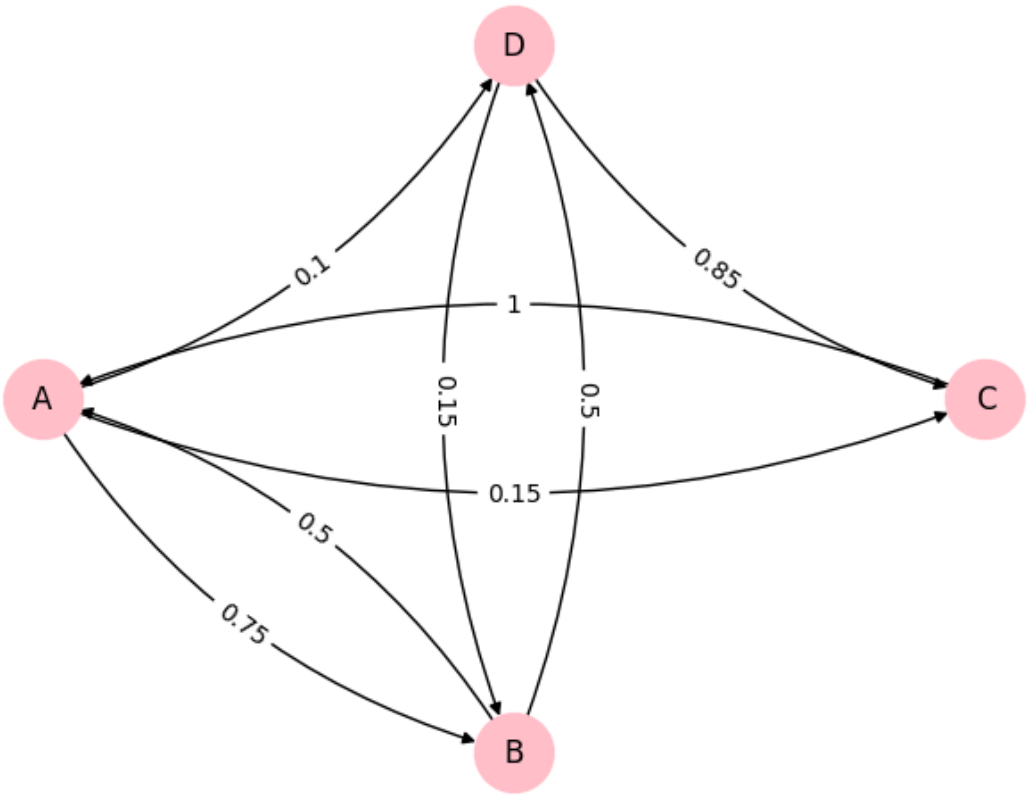
In [13]:
```
# Create a Vertex DataFrame with unique ID column "id"
#v = sqlContext.createDataFrame('write code here')
# Create a directed GraphFrame
#DG=nx.DiGraph()
# Create a multi-edge directed GraphFrame
MG = nx.MultiDiGraph()

#MG.add_node("c4",name="C4-page")
#Add nodes (vertices) in here
MG.add_node("A", name = "A")
MG.add_node("B", name = "B")
MG.add_node("C", name = "C")
MG.add_node("D", name = "D")


MG.add_weighted_edges_from([
    ('A', 'B', 0.75), ('A', 'C', 0.15), ('A', 'D', 0.1),
    ('B', 'D', 1/2), ('B', 'A', 1/2),
    ('C', 'A', 1),
    ('D','C', 0.85), ('D', 'B', 0.15)
])


#collect labels
list_nodes=list(MG.nodes)
labes=[]
for i in range(0,len(MG)):
    au=list_nodes[i]
    bu=MG.nodes[au]['name']
    labes.append(au)
    labes.append(bu)
print(labes)

labels = {labes[i]: labes[i + 1] for i in range(0, len(labes), 2)}
edge_labels = nx.get_edge_attributes(MG,'weight')
#plot figure, note different commands
pos = nx.shell_layout(MG)
nx.draw(MG, pos,node_color='pink', connectionstyle="arc3, rad=0.2", with_labels=
```

```
nx.draw_networkx_labels(MG, pos,labels=labels)
nx.draw_networkx_edge_labels(MG, pos, connectionstyle="arc3, rad=0.2", edge_labe
```

```
['A', 'A', 'B', 'B', 'C', 'C', 'D', 'D']
```

Out[13]:

```
{('A', 'B', 0): Text(-0.5749998780213402, -0.6333334880941961, '0.75'),
 ('A', 'C', 0): Text(6.7577307445443324e-06, -0.2666666229422878, '0.15'),
 ('A', 'D', 0): Text(-0.425000052866945, 0.36666677745029896, '0.1'),
 ('B', 'D', 0): Text(0.14999983706322273, 9.839889808294089e-06, '0.5'),
 ('B', 'A', 0): Text(-0.425000110053779, -0.3666665993285805, '0.5'),
 ('C', 'A', 0): Text(-6.757730744322288e-06, 0.2666667103650644, '1'),
 ('D', 'C', 0): Text(0.42499995368984456, 0.3666665995944638, '0.85'),
 ('D', 'B', 0): Text(-0.15000016290458018, -9.839889808072044e-06, '0.15')}
```



Below is the code for applying the pagerank algorithm to the graph.

3. What the pagerank algorithm computes?

PageRank computes the probability of a web surfer to end up at a certain webpage (represented as a node) if they were to follow a certain pattern of movements represented as a transition matrix M that contains the probability of taking a path from a node to another.

4. Calculate PageRank for the matrix in figure 5.3 of the course book change damping from 0.85, to 0.5 and 0.9, for example in the loop or manually changing the parameter vvalue. Explain what the 'damping' parameter does? How did the pageRank results change in this case?

Damping parameter is used to combat dead ends (node C), the number represents the chance of a random surfer to "teleport" to a random page rather than following the

traditional transition matrix, reducing the effects of all surfers ending up in the dead end. The parameter made the probabilities of the nodes to be roughly the same after 100 iterations, which would not be the case if there was not damping parameter.

In [16]:
```
MG = nx.MultiDiGraph()


MG.add_node("A", name = "A")
MG.add_node("B", name = "B")
MG.add_node("C", name = "C")
MG.add_node("D", name = "D")


MG.add_weighted_edges_from([
    ('A', 'B', 1/3), ('A', 'C', 1/3), ('A', 'D', 1/3),
    ('B', 'A', 1/2), ('B', 'D', 1/2),
    ('D','C', 1/2), ('D', 'B', 1/2)
])

for i in [0.85, 0.5, 0.9]:
    # Run PageRank algorithm, and show results.
    results = nx.pagerank(MG, alpha=i,max_iter=100,weight='weight') #alpha is a
    print(f"{i} dampening: {results}")
```

```
0.85 dampening: {'A': 0.2061857491852643, 'B': 0.2646047502715787, 'C': 0.2646047
502715787, 'D': 0.2646047502715787}
0.5 dampening: {'A': 0.22222232818603516, 'B': 0.2592592239379883, 'C': 0.2592592
239379883, 'D': 0.2592592239379883}
0.9 dampening: {'A': 0.20408193426364896, 'B': 0.26530602191211694, 'C': 0.265306
02191211694, 'D': 0.26530602191211694}
```

Task 4.2.2 (4 p)

In the code below, the Moodle zip-package data is used.

1. Describe the data, what information do the files contain (you can have a look at the readme-Ego.txt file available on the web page https://snap.stanford.edu/data/egonets-Facebook.html)? We do not use the feature files in this exercise.

nodeId.edges : The edges in the ego network for the node 'nodeId'. Edges are undirected for facebook, and directed (a follows b) for twitter and gplus. The 'ego' node does not appear, but it is assumed that they follow every node id that appears in this file.

nodeId.circles : The set of circles for the ego node. Each line contains one circle, consisting of a series of node ids. The first entry in each line is the name of the circle.

nodeId.feat : The features for each of the nodes that appears in the edge file.

nodeId.egofeat : The features for the ego user.

nodeId.featnames : The names of each of the feature dimensions. Features are '1' if the user has this property in their profile, and '0' otherwise. This file has been anonymized for facebook users, since the names of the features would reveal private data.

2. What is the degree of a node?

The degree of a node is the amount of edges connected to it

3. Is the PageRank algorithm any better than just the degree of a node to find the most important vertices of the network?

If we wish to only know what the most important vertices are, the degree of a node is definitely viable, however what the PageRank also provides is the probability density of the entire network, so we have a clearer picture of the behavior of the network.

4. Give some examples which user ids you think are important by visual inspection (see image), and in pagerank or degree. (For some analyses a vertex can be important if it connects network parts that would be isolated without the connecting vertex.)

Vertices that connect outliers: 769, 811, 866, 879 Verticies that appear "important" according to the images: 769, 856, 851, etc.

In [17]:
```
import networkx as nx
from pyspark.sql.functions import col
import matplotlib.pyplot as plt
import pyspark.sql.functions as F
from pyspark.sql.window import Window as W
import numpy as np
#read graph edges(or arcs, lines) (multiple edges per vertex can exist) from "69
lines = sc.textFile("698.edges")
edges = lines.map(lambda l: l.split(" ")). \
            map(lambda p: Row( src=int(p[0]), dst=int(p[1])) )
edges = sqlContext.createDataFrame(edges)

#read graph vertices(or nodes, points) (these are unique) from "698.feat"
lines = sc.textFile("698.feat")
vertices = lines.map(lambda l: l.split(" ")). \
            map(lambda p: Row(id=int(p[0]), name="userid_"+p[0]) )
vertices = sqlContext.createDataFrame(vertices)

idx_array = vertices.select("id").rdd.flatMap(lambda x: x).collect()
#print(idx_array)
print("Edges")
edges.show()
print("Vertices")
vertices.show()
len_vertices=vertices.count()
print(len_vertices)
print(edges.count())

list_edges=[]
for i in edges.collect():
   list_edges.append(tuple(i))
print("List_edges")
print(list_edges)
#print(len(list_edges))
#print(len_vertices)

#MG = nx.MultiDiGraph()
DG = nx.DiGraph()
for x in edges.collect():
```

```
    DG.add_edges_from([(x[0],x[1])], weight=1)
for x in vertices.collect():
    DG.add_node(x[0],label=x[1])
#g = ig.Graph(len_vertices,edges=list_edges,directed=True) #,vertex_name_attr='i
#g.vs["name"] = list(vertices.select("name").collect())
#g.vs["id"] = list(vertices.select("id").collect())

G = nx.path_graph(DG)  # or DiGraph, MultiGraph, MultiDiGraph, etc
print("Degrees of nodes")
print(G.degree)

#Get degrees of nodes in Graph
degree_list=[]
for i in idx_array:
    degree_list.append(G.degree[i])
print(degree_list)
#Get pageranks of Graph
pagerank=nx.pagerank(DG, alpha=0.85,max_iter=100)
print("pagerank")
#### Show pagerank and degree results with nodes
print(pagerank)  #showing only pagerank to starting indices
#combine all data in the same dataframe using a,b,c and d
a = vertices.persist()
print("a")
a.show()
b = sqlContext.createDataFrame([(l,pagerank[l]) for l in pagerank.keys()], ['idx
print("b")
b.show()

c = sqlContext.createDataFrame([(l,) for l in degree_list], ['degree'])
print("c")
c.show()
a = a.withColumn("idx2", F.monotonically_increasing_id())
print("a")
print(a)
b = b.withColumn("idx", F.monotonically_increasing_id())
print("b")
b.show()
c = c.withColumn("idx", F.monotonically_increasing_id())
print("c")
c.show()

windowSpec = W.orderBy("idx")
windowSpec2 = W.orderBy("idx2")
a = a.withColumn("idx2", F.row_number().over(windowSpec2))
b = b.withColumn("idx", F.row_number().over(windowSpec))
c = c.withColumn("idx", F.row_number().over(windowSpec))

d = a.join(b, a.idx2 == b.idx).drop("idx2")
print("d")
d.show()

result=d.join(c,d.idx == c.idx).drop("idx")
result.orderBy(result.pagerank.desc())
#final combined dataframe
print("Result")
result.show()

#get positions
DG_pr = nx.DiGraph()
```

```
for x in edges.collect():
    DG_pr.add_edges_from([(x[0],x[1])], weight=1)
for x in result.collect():
    DG_pr.add_node(x[0],pagerank=x[2],degree=x[3])
#pagerank
pos = nx.spring_layout(DG)
#determine node size by pagerank value, exxagerate it with ((size*570)**2.2)
size_nde={}
print("DG_pr.nodes")
for x in DG_pr.nodes():
    #print(DG_pr.nodes[x]['pagerank'])
    aux=int((DG_pr.nodes[x]['pagerank']*570)**2.2)
    if aux<20:
        aux=20
    size_nde.update({x: aux})
#plot the pagerank figure
print("Size_nde")
print(size_nde)
print(len(DG))
print(len(size_nde))
pos = nx.kamada_kawai_layout(DG)
plt.figure(figsize=(10, 10))
nx.draw_networkx_nodes(DG, pos, node_size=[size_nde[node] for node in DG.nodes()
nx.draw_networkx_edges(DG, pos)
nx.draw_networkx_labels(DG, pos)
plt.title("698 Network with pagerank (different sizes)")
plt.axis("off")
plt.show()

#degree
pos = nx.spring_layout(DG)
#determine node size by degree value, exxagerate it with (size*500)
size_nde={}
print("DG_pr.nodes")
for x in DG_pr.nodes():
    #print(DG_pr.nodes[x]['degree'])
    aux=int((DG_pr.nodes[x]['degree']*500))
    if aux<20:
        aux=20
    size_nde.update({x: aux})
#plot the degree figure
print("Size_nde")
print(size_nde)
print(len(DG))
print(len(size_nde))
pos = nx.kamada_kawai_layout(DG)
plt.figure(figsize=(10, 10))
nx.draw_networkx_nodes(DG, pos, node_size=[size_nde[node] for node in DG.nodes()
nx.draw_networkx_edges(DG, pos)
nx.draw_networkx_labels(DG, pos)
plt.title("698 Network with degree (different sizes)")
plt.axis("off")
plt.show()

## get different communities
communities_generator = nx.community.girvan_newman(DG)
top_level_communities = next(communities_generator)
colors = plt.cm.RdYlBu(np.linspace(0, 1, len(top_level_communities)))
#Get colors for communities
node_colors = {}
```

```
for i, community in enumerate(top_level_communities):
    #print(community)
    for node in community:
        #print(node)
        node_colors[node] = colors[i]
print("Node colors")
print(node_colors)
#get positions
pos = nx.spring_layout(DG,iterations=10)

#plot communities
plt.figure(figsize=(11, 11))
nx.draw_networkx_nodes(DG, pos, node_color=[node_colors[node] for node in DG.nod
nx.draw_networkx_edges(DG, pos)
nx.draw_networkx_labels(DG, pos)
plt.title("698 Network with Girvan-Newman Communities (different colors)")
plt.axis("off")
plt.show()

## Alternative
# Visualize the first level of communities ahainst the rest
pos = nx.spring_layout(G)
plt.figure(figsize=(11, 11))

# Color nodes by their community
node_colors = ['skyblue' if node in top_level_communities[0] else 'orange' for n
nx.draw(DG, pos, with_labels=True, node_color=node_colors, font_size=12, node_si

plt.title("698 Network with Girvan-Newman Communities, top communities blue, res
plt.axis("off")
plt.show()
#Show detected communities
communities_generator = nx.community.girvan_newman(DG)
top_level_communities = next(communities_generator)
print("Detected Communities:", top_level_communities)
```

```
Edges
+---+---+
|src|dst|
+---+---+
|881|858|
|828|697|
|884|864|
|856|869|
|889|856|
|872|873|
|719|713|
|861|863|
|840|803|
|864|856|
|719|840|
|745|805|
|865|857|
|874|873|
|708|823|
|872|885|
|870|871|
|888|889|
|825|889|
|869|774|
+---+---+
only showing top 20 rows
Vertices
+---+----------+
| id|      name|
+---+----------+
|810|userid_810|
|857|userid_857|
|811|userid_811|
|858|userid_858|
|859|userid_859|
|860|userid_860|
|769|userid_769|
|861|userid_861|
|840|userid_840|
|862|userid_862|
|863|userid_863|
|729|userid_729|
|864|userid_864|
|865|userid_865|
|866|userid_866|
|867|userid_867|
|697|userid_697|
|868|userid_868|
|869|userid_869|
|708|userid_708|
+---+----------+
only showing top 20 rows
66
540
List_edges
[(881, 858), (828, 697), (884, 864), (856, 869), (889, 856), (872, 873), (719, 71
3), (861, 863), (840, 803), (864, 856), (719, 840), (745, 805), (865, 857), (874,
873), (708, 823), (872, 885), (870, 871), (888, 889), (825, 889), (869, 774), (72
9, 888), (769, 869), (823, 840), (861, 729), (774, 769), (805, 800), (713, 819),
(774, 856), (885, 859), (729, 876), (872, 887), (895, 881), (697, 890), (823, 81
```

0), (870, 895), (871, 879), (823, 753), (819, 719), (708, 869), (719, 810), (745,
830), (747, 830), (774, 810), (856, 747), (879, 881), (776, 874), (774, 805), (74
7, 819), (772, 828), (888, 804), (890, 753), (889, 886), (856, 864), (864, 893),
(747, 719), (861, 889), (819, 708), (769, 811), (867, 874), (887, 874), (868, 85
7), (828, 810), (878, 889), (889, 753), (719, 772), (819, 823), (867, 877), (819,
828), (876, 889), (708, 819), (828, 856), (856, 708), (800, 810), (708, 830), (72
9, 882), (887, 776), (867, 776), (719, 774), (861, 804), (856, 878), (772, 713),
(894, 879), (810, 747), (772, 805), (774, 719), (753, 708), (697, 830), (772, 85
6), (885, 877), (886, 889), (745, 810), (729, 804), (840, 823), (887, 872), (893,
856), (697, 753), (810, 823), (804, 863), (805, 713), (713, 823), (830, 800), (88
1, 871), (890, 708), (830, 745), (866, 881), (863, 804), (800, 805), (840, 708),
(859, 885), (828, 880), (868, 865), (889, 825), (745, 747), (884, 825), (888, 86
1), (872, 867), (889, 729), (830, 713), (747, 713), (856, 804), (856, 825), (871,
857), (800, 719), (886, 863), (703, 828), (866, 879), (800, 774), (819, 880), (80
5, 830), (753, 769), (772, 800), (697, 800), (708, 713), (823, 719), (819, 753),
(856, 840), (886, 729), (876, 886), (804, 876), (708, 747), (873, 776), (769, 82
8), (885, 872), (713, 774), (885, 873), (869, 856), (811, 856), (800, 697), (823,
805), (885, 776), (747, 810), (697, 823), (703, 819), (811, 800), (753, 729), (77
4, 811), (879, 894), (825, 884), (747, 774), (870, 894), (713, 805), (884, 893),
(729, 856), (747, 804), (747, 769), (745, 800), (774, 708), (810, 856), (856, 77
2), (869, 811), (703, 747), (877, 867), (811, 890), (856, 888), (867, 872), (703,
805), (719, 697), (800, 713), (713, 810), (729, 863), (888, 876), (856, 863), (88
9, 888), (876, 856), (823, 830), (747, 803), (886, 878), (774, 869), (776, 867),
(811, 769), (873, 872), (857, 868), (869, 890), (719, 819), (889, 878), (840, 83
0), (828, 830), (697, 828), (774, 747), (747, 697), (830, 708), (819, 747), (871,
895), (776, 873), (856, 810), (864, 825), (828, 703), (774, 800), (708, 769), (76
9, 800), (800, 747), (823, 703), (890, 811), (803, 747), (890, 769), (856, 769),
(887, 867), (697, 819), (840, 697), (753, 856), (697, 772), (886, 804), (865, 86
8), (729, 753), (811, 708), (697, 719), (819, 703), (830, 840), (823, 708), (697,
745), (708, 753), (894, 870), (708, 719), (805, 880), (881, 879), (697, 856), (82
3, 856), (747, 823), (828, 819), (708, 697), (774, 745), (878, 888), (872, 776),
(747, 805), (800, 823), (810, 800), (803, 840), (804, 882), (747, 856), (878, 88
6), (753, 890), (830, 697), (769, 890), (866, 895), (828, 823), (828, 745), (882,
729), (890, 869), (769, 774), (708, 811), (856, 823), (880, 823), (804, 888), (83
0, 719), (861, 888), (719, 805), (697, 747), (880, 819), (776, 872), (830, 856),
(882, 863), (863, 856), (747, 880), (880, 805), (745, 697), (774, 823), (810, 71
9), (810, 713), (856, 800), (880, 828), (769, 856), (828, 800), (840, 828), (869,
769), (828, 708), (819, 774), (772, 719), (769, 798), (870, 858), (713, 840), (71
9, 745), (776, 887), (823, 800), (810, 745), (885, 874), (819, 805), (878, 856),
(825, 864), (876, 882), (819, 840), (870, 881), (873, 874), (856, 828), (878, 80
4), (867, 885), (885, 887), (830, 747), (713, 719), (810, 828), (805, 819), (856,
697), (805, 747), (804, 878), (823, 713), (878, 729), (861, 753), (745, 823), (87
8, 861), (800, 745), (828, 772), (823, 745), (895, 858), (873, 885), (713, 697),
(804, 856), (878, 863), (882, 889), (880, 803), (881, 870), (863, 878), (708, 84
0), (872, 874), (774, 828), (828, 747), (713, 747), (856, 889), (823, 697), (708,
774), (719, 708), (856, 830), (882, 876), (863, 876), (879, 866), (747, 828), (75
3, 823), (863, 886), (703, 880), (745, 713), (856, 703), (823, 747), (823, 828),
(800, 811), (769, 697), (874, 872), (753, 697), (895, 866), (840, 747), (879, 87
0), (873, 867), (828, 769), (828, 713), (868, 862), (819, 830), (880, 747), (874,
885), (719, 828), (863, 861), (713, 828), (830, 823), (830, 805), (840, 713), (74
5, 774), (753, 747), (869, 708), (886, 876), (895, 870), (887, 885), (800, 769),
(888, 878), (747, 745), (811, 869), (828, 774), (697, 708), (876, 804), (863, 88
2), (862, 865), (708, 800), (772, 697), (803, 880), (810, 774), (753, 889), (713,
830), (856, 811), (857, 865), (840, 774), (774, 840), (697, 805), (729, 886), (80
0, 828), (719, 823), (747, 703), (703, 856), (804, 861), (858, 871), (874, 776),
(881, 866), (862, 868), (800, 708), (708, 856), (769, 753), (865, 862), (805, 82
3), (805, 810), (856, 861), (840, 856), (729, 861), (862, 857), (719, 703), (828,
805), (745, 828), (857, 871), (713, 772), (719, 830), (856, 774), (800, 772), (88
1, 895), (893, 884), (697, 840), (882, 856), (885, 867), (713, 800), (804, 729),
(863, 729), (747, 840), (895, 871), (856, 893), (876, 861), (856, 876), (864, 88

```
4), (863, 888), (867, 887), (774, 713), (828, 840), (888, 856), (830, 828), (708,
890), (861, 856), (870, 879), (876, 878), (894, 866), (876, 888), (889, 876), (85
7, 862), (879, 871), (863, 889), (890, 697), (729, 878), (747, 753), (805, 719),
(858, 870), (745, 719), (713, 745), (819, 856), (805, 774), (798, 769), (708, 82
8), (729, 889), (800, 830), (830, 774), (769, 708), (776, 885), (886, 882), (867,
873), (747, 708), (819, 697), (861, 878), (893, 864), (697, 713), (805, 703), (77
4, 819), (753, 861), (858, 895), (703, 823), (856, 882), (874, 867), (877, 885),
(805, 697), (719, 747), (805, 828), (747, 800), (823, 880), (804, 747), (804, 88
6), (805, 840), (713, 708), (811, 774), (889, 882), (878, 876), (825, 856), (888,
863), (823, 819), (880, 703), (882, 886), (877, 776), (805, 745), (776, 877), (87
1, 881), (819, 713), (856, 753), (719, 800), (876, 729), (840, 819), (861, 876),
(804, 889), (889, 804), (828, 719), (871, 858), (889, 861), (840, 805), (769, 74
7), (805, 772), (888, 729), (874, 887), (840, 719), (703, 719), (753, 819), (871,
870), (810, 805), (866, 894), (889, 863), (876, 863), (697, 769), (882, 804), (85
6, 729), (856, 819), (774, 830), (800, 856), (858, 881), (823, 774), (830, 819)]
Degrees of nodes
[(881, 1), (858, 2), (828, 2), (697, 2), (884, 2), (864, 2), (856, 2), (869, 2),
(889, 2), (872, 2), (873, 2), (719, 2), (713, 2), (861, 2), (863, 2), (840, 2),
(803, 2), (745, 2), (805, 2), (865, 2), (857, 2), (874, 2), (708, 2), (823, 2),
(885, 2), (870, 2), (871, 2), (888, 2), (825, 2), (774, 2), (729, 2), (769, 2),
(800, 2), (819, 2), (859, 2), (876, 2), (887, 2), (895, 2), (890, 2), (810, 2),
(879, 2), (753, 2), (830, 2), (747, 2), (776, 2), (772, 2), (804, 2), (886, 2),
(893, 2), (811, 2), (867, 2), (868, 2), (878, 2), (877, 2), (882, 2), (894, 2),
(866, 2), (880, 2), (703, 2), (798, 2), (862, 2), (860, 2), (875, 2), (883, 2),
(891, 2), (892, 1)]
[2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
2, 2, 2, 2, 1, 2, 2, 2, 2, 2, 2, 2]
pagerank
{881: 0.0200372578886205, 858: 0.013941304862104563, 828: 0.024294863650176406, 6
97: 0.02213609227358657, 884: 0.009285042614420378, 864: 0.010837320553364706, 85
6: 0.041772440128214725, 869: 0.010005345284208041, 889: 0.019546831370734104, 87
2: 0.017601650597721636, 873: 0.014996431263644201, 719: 0.02041676355779303, 71
3: 0.019188912259543344, 861: 0.014520048016190718, 863: 0.01619518580157567, 84
0: 0.017435801157622766, 803: 0.005989469613922507, 745: 0.014537252069404948, 80
5: 0.020595268400295547, 865: 0.014119902463922265, 857: 0.01735262303891543, 87
4: 0.017601650597721636, 708: 0.02253966583491923, 823: 0.0229559823982319, 885:
0.024603838463985058, 870: 0.02017155084751108, 871: 0.02060794955652292, 888: 0.
013243933725673887, 825: 0.009972446738338199, 774: 0.02216007767093775, 729: 0.0
17462670639124753, 769: 0.018932788879603413, 800: 0.020843301420921008, 819: 0.0
19551285124572795, 859: 0.005043307756251917, 876: 0.01619518580157567, 887: 0.01
4996431263644201, 895: 0.017042884725160634, 890: 0.010018196659006624, 810: 0.01
3534105445490748, 879: 0.017270234651913498, 753: 0.016050438033102944, 830: 0.01
801558077546333, 747: 0.027312330430353842, 776: 0.020448190474758227, 772: 0.010
220244993035477, 804: 0.017337520652445277, 886: 0.012099345694712497, 893: 0.008
587722853585488, 811: 0.01117325467473895, 867: 0.020448190474758227, 868: 0.0141
19902463922265, 878: 0.014729371730788471, 877: 0.010009296961765913, 882: 0.0119
50566726922835, 894: 0.011258563442740606, 866: 0.01428990740041199, 880: 0.01082
931512737666, 703: 0.01153113758527739, 798: 0.003770172984974192, 862: 0.0141199
02463922265, 860: 0.0024291497975708507, 875: 0.0024291497975708507, 883: 0.00242
91497975708507, 891: 0.0024291497975708507, 892: 0.0024291497975708507}
a
+---+----------+
| id|      name|
+---+----------+
|810|userid_810|
|857|userid_857|
|811|userid_811|
|858|userid_858|
|859|userid_859|
```

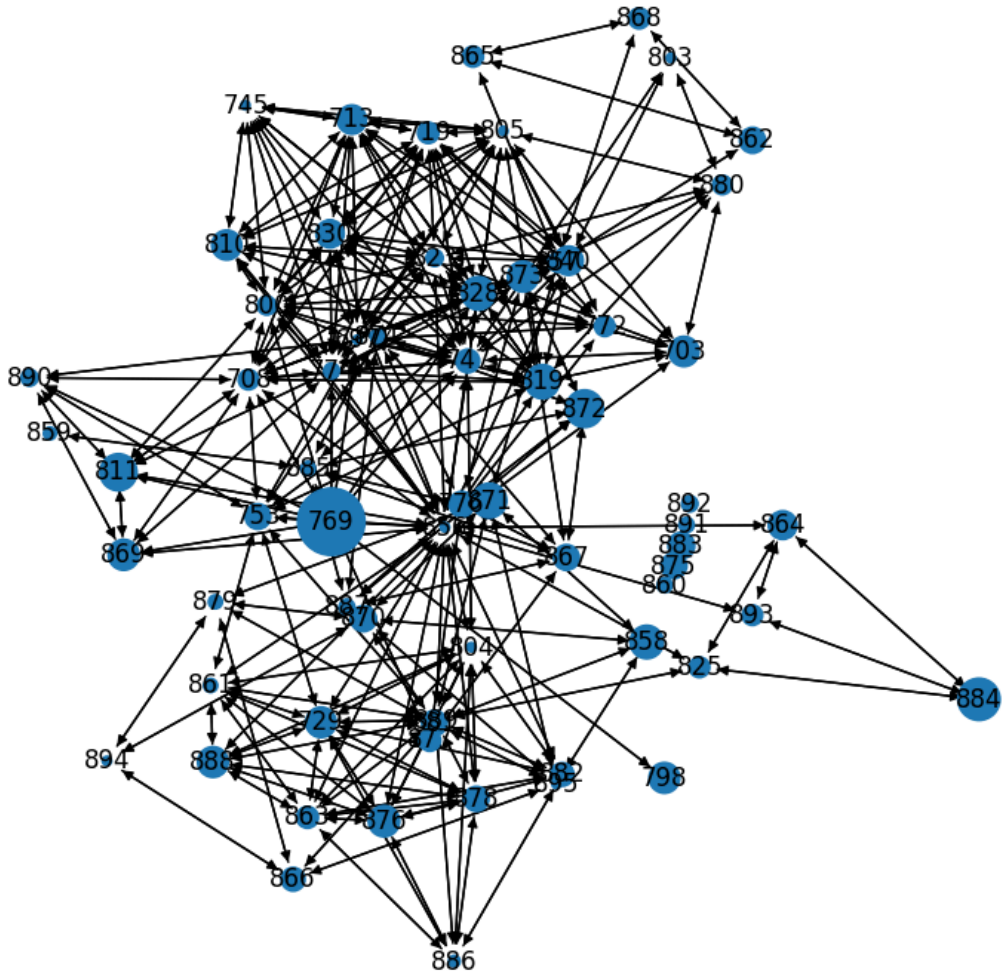
```
|860|userid_860|
|769|userid_769|
|861|userid_861|
|840|userid_840|
|862|userid_862|
|863|userid_863|
|729|userid_729|
|864|userid_864|
|865|userid_865|
|866|userid_866|
|867|userid_867|
|697|userid_697|
|868|userid_868|
|869|userid_869|
|708|userid_708|
+---+----------+
only showing top 20 rows
b
+---+--------------------+
|idx|            pagerank|
+---+--------------------+
|881|  0.0200372578886205|
|858|0.013941304862104563|
|828|0.024294863650176406|
|697| 0.02213609227358657|
|884|0.009285042614420378|
|864|0.010837320553364706|
|856|0.041772440128214725|
|869|0.010005345284208041|
|889|0.019546831370734104|
|872|0.017601650597721636|
|873|0.014996431263644201|
|719| 0.02041676355779303|
|713|0.019188912259543344|
|861|0.014520048016190718|
|863| 0.01619518580157567|
|840|0.017435801157622766|
|803|0.005989469613922507|
|745|0.014537252069404948|
|805|0.020595268400295547|
|865|0.014119902463922265|
+---+--------------------+
only showing top 20 rows
c
+------+
|degree|
+------+
|     2|
|     2|
|     2|
|     2|
|     2|
|     2|
|     2|
|     2|
|     2|
|     2|
|     2|
|     2|
|     2|
```

```
|     2|
|     2|
|     2|
|     2|
|     2|
|     2|
|     2|
+------+
only showing top 20 rows
a
DataFrame[id: bigint, name: string, idx2: bigint]
b
+-----------+--------------------+
|        idx|            pagerank|
+-----------+--------------------+
|          0|  0.0200372578886205|
|          1|0.013941304862104563|
|          2|0.024294863650176406|
|          3| 0.02213609227358657|
|          4|0.009285042614420378|
| 8589934592|0.010837320553364706|
| 8589934593|0.041772440128214725|
| 8589934594|0.010005345284208041|
| 8589934595|0.019546831370734104|
| 8589934596|0.017601650597721636|
|17179869184|0.014996431263644201|
|17179869185| 0.02041676355779303|
|17179869186|0.019188912259543344|
|17179869187|0.014520048016190718|
|17179869188| 0.01619518580157567|
|25769803776|0.017435801157622766|
|25769803777|0.005989469613922507|
|25769803778|0.014537252069404948|
|25769803779|0.020595268400295547|
|25769803780|0.014119902463922265|
+-----------+--------------------+
only showing top 20 rows
c
+------+-----------+
|degree|        idx|
+------+-----------+
|     2|          0|
|     2|          1|
|     2|          2|
|     2|          3|
|     2|          4|
|     2| 8589934592|
|     2| 8589934593|
|     2| 8589934594|
|     2| 8589934595|
|     2| 8589934596|
|     2|17179869184|
|     2|17179869185|
|     2|17179869186|
|     2|17179869187|
|     2|17179869188|
|     2|25769803776|
|     2|25769803777|
|     2|25769803778|
|     2|25769803779|
```

```
|     2|25769803780|
+------+-----------+
only showing top 20 rows
d
+---+----------+---+--------------------+
| id|      name|idx|            pagerank|
+---+----------+---+--------------------+
|810|userid_810|  1|  0.0200372578886205|
|857|userid_857|  2|0.013941304862104563|
|811|userid_811|  3|0.024294863650176406|
|858|userid_858|  4| 0.02213609227358657|
|859|userid_859|  5|0.009285042614420378|
|860|userid_860|  6|0.010837320553364706|
|769|userid_769|  7|0.041772440128214725|
|861|userid_861|  8|0.010005345284208041|
|840|userid_840|  9|0.019546831370734104|
|862|userid_862| 10|0.017601650597721636|
|863|userid_863| 11|0.014996431263644201|
|729|userid_729| 12| 0.02041676355779303|
|864|userid_864| 13|0.019188912259543344|
|865|userid_865| 14|0.014520048016190718|
|866|userid_866| 15| 0.01619518580157567|
|867|userid_867| 16|0.017435801157622766|
|697|userid_697| 17|0.005989469613922507|
|868|userid_868| 18|0.014537252069404948|
|869|userid_869| 19|0.020595268400295547|
|708|userid_708| 20|0.014119902463922265|
+---+----------+---+--------------------+
only showing top 20 rows
Result
+---+----------+--------------------+------+
| id|      name|            pagerank|degree|
+---+----------+--------------------+------+
|810|userid_810|  0.0200372578886205|     2|
|857|userid_857|0.013941304862104563|     2|
|811|userid_811|0.024294863650176406|     2|
|858|userid_858| 0.02213609227358657|     2|
|859|userid_859|0.009285042614420378|     2|
|860|userid_860|0.010837320553364706|     2|
|769|userid_769|0.041772440128214725|     2|
|861|userid_861|0.010005345284208041|     2|
|840|userid_840|0.019546831370734104|     2|
|862|userid_862|0.017601650597721636|     2|
|863|userid_863|0.014996431263644201|     2|
|729|userid_729| 0.02041676355779303|     2|
|864|userid_864|0.019188912259543344|     2|
|865|userid_865|0.014520048016190718|     2|
|866|userid_866| 0.01619518580157567|     2|
|867|userid_867|0.017435801157622766|     2|
|697|userid_697|0.005989469613922507|     2|
|868|userid_868|0.014537252069404948|     2|
|869|userid_869|0.020595268400295547|     2|
|708|userid_708|0.014119902463922265|     2|
+---+----------+--------------------+------+
only showing top 20 rows
DG_pr.nodes
Size_nde
{881: 153, 858: 264, 828: 264, 697: 20, 884: 419, 864: 193, 856: 20, 869: 225, 88
9: 46, 872: 333, 873: 225, 719: 107, 713: 201, 861: 46, 863: 112, 840: 201, 803:
20, 745: 20, 805: 20, 865: 104, 857: 95, 874: 45, 708: 98, 823: 69, 885: 48, 870:
```

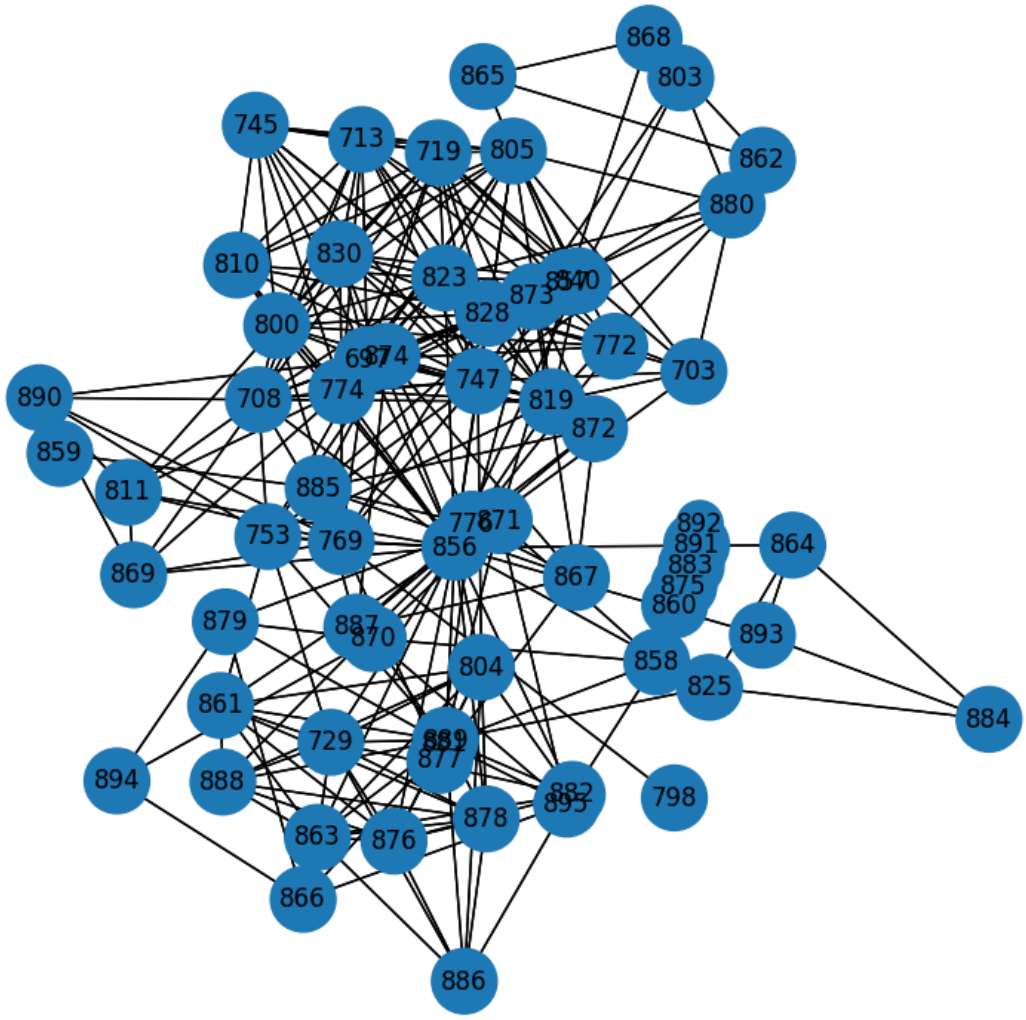
```
159, 871: 286, 888: 222, 825: 100, 774: 59, 729: 221, 769: 1068, 800: 85, 819: 27
5, 859: 39, 876: 231, 887: 58, 895: 20, 890: 68, 810: 212, 879: 46, 753: 154, 83
0: 187, 747: 132, 776: 154, 772: 98, 804: 20, 886: 32, 893: 98, 811: 324, 867: 15
6, 868: 104, 878: 148, 877: 112, 882: 130, 894: 20, 866: 132, 880: 89, 703: 215,
798: 222, 862: 159, 860: 54, 875: 156, 883: 168, 891: 54, 892: 62}
66
66
```

698 Network with pagerank (different sizes)




```
DG_pr.nodes
Size_nde
{881: 500, 858: 1000, 828: 1000, 697: 1000, 884: 1000, 864: 1000, 856: 1000, 869:
1000, 889: 1000, 872: 1000, 873: 1000, 719: 1000, 713: 1000, 861: 1000, 863: 100
0, 840: 1000, 803: 1000, 745: 1000, 805: 1000, 865: 1000, 857: 1000, 874: 1000, 7
08: 1000, 823: 1000, 885: 1000, 870: 1000, 871: 1000, 888: 1000, 825: 1000, 774:
1000, 729: 1000, 769: 1000, 800: 1000, 819: 1000, 859: 1000, 876: 1000, 887: 100
0, 895: 1000, 890: 1000, 810: 1000, 879: 1000, 753: 1000, 830: 1000, 747: 1000, 7
76: 1000, 772: 1000, 804: 1000, 886: 1000, 893: 1000, 811: 1000, 867: 1000, 868:
1000, 878: 1000, 877: 1000, 882: 1000, 894: 1000, 866: 1000, 880: 1000, 703: 100
0, 798: 1000, 862: 1000, 860: 1000, 875: 1000, 883: 1000, 891: 1000, 892: 500}
66
66
```

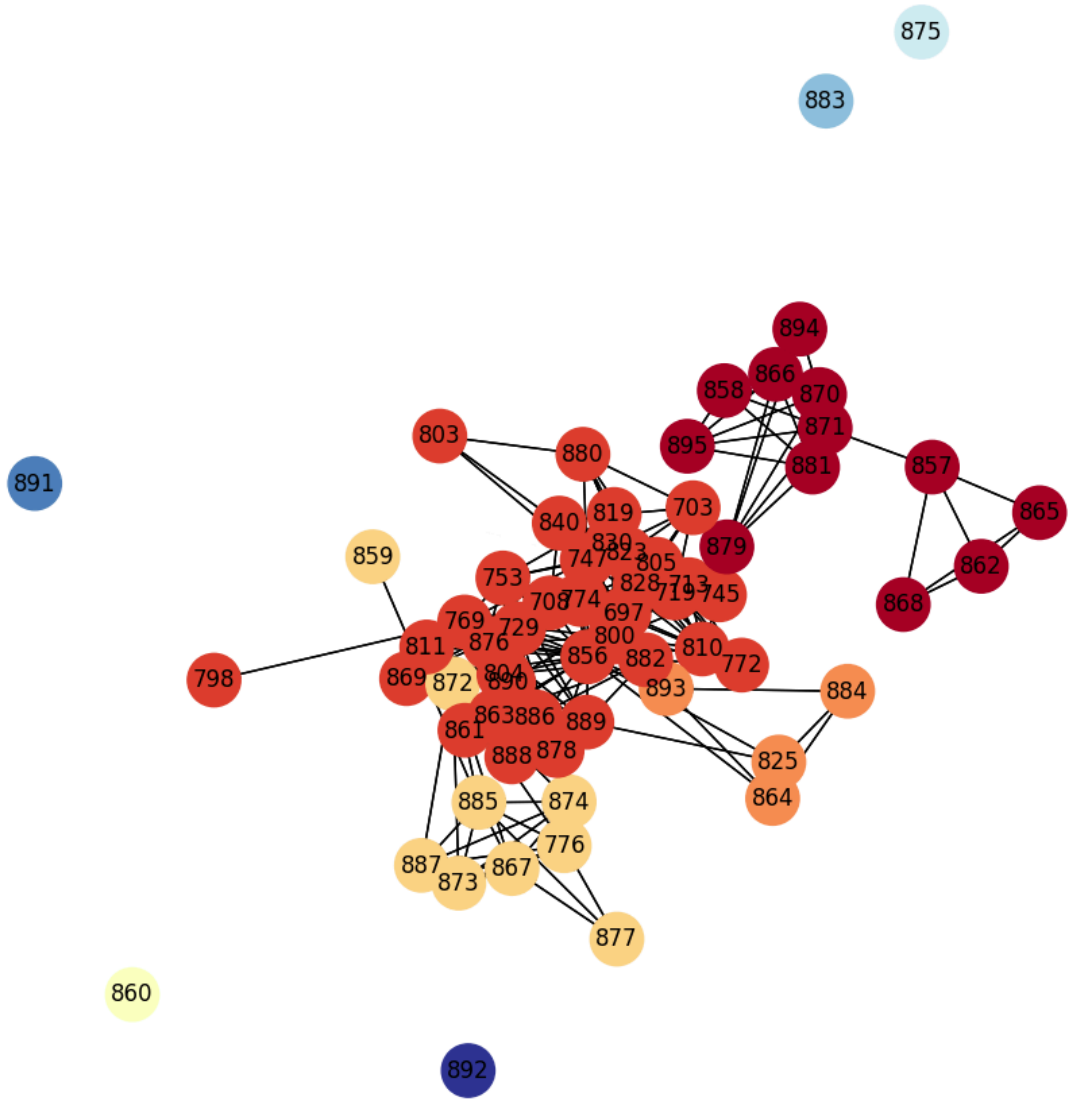
698 Network with degree (different sizes)



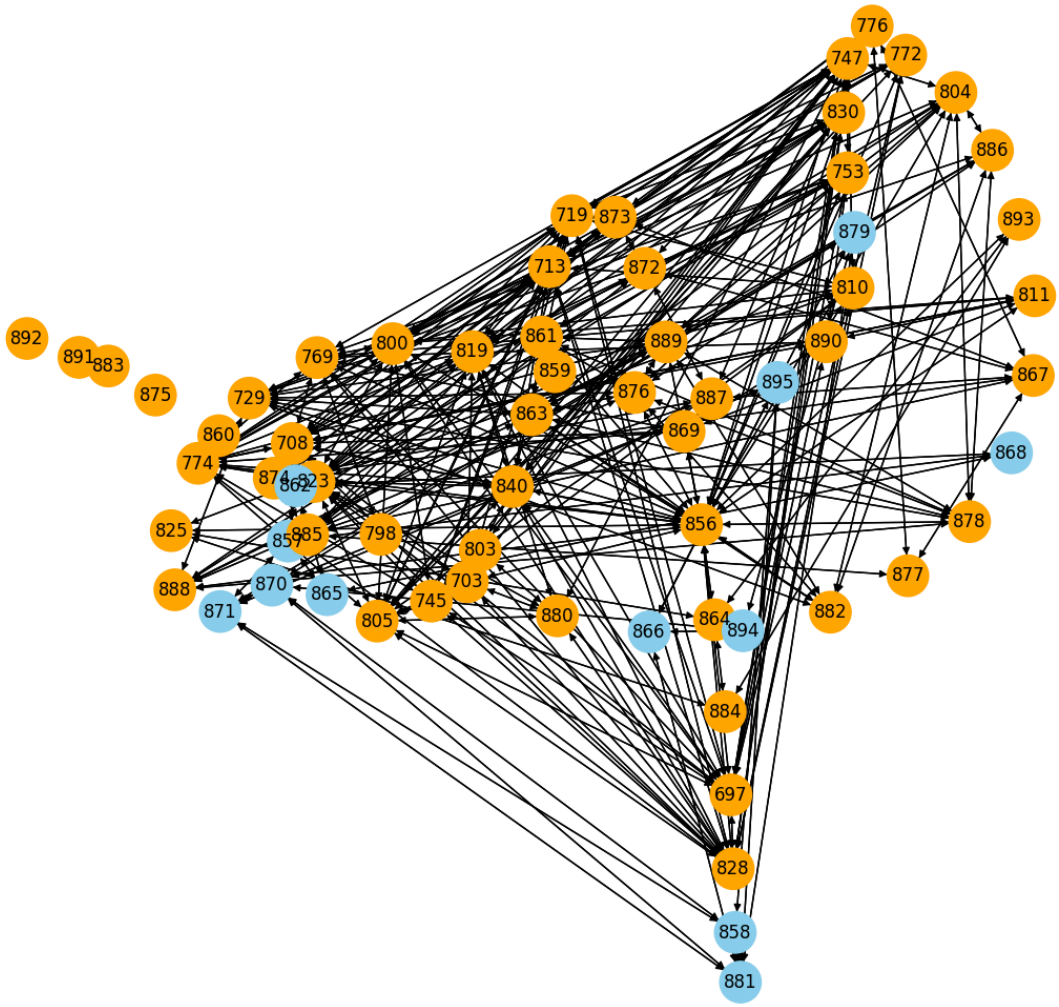
Node colors

```
{865: array([0.64705882, 0.        , 0.14901961, 1.        ]), 866: array([0.6470
5882, 0.        , 0.14901961, 1.        ]), 868: array([0.64705882, 0.        ,
0.14901961, 1.        ]), 870: array([0.64705882, 0.        , 0.14901961, 1.
]), 871: array([0.64705882, 0.        , 0.14901961, 1.        ]), 879: array([0.6
4705882, 0.        , 0.14901961, 1.        ]), 881: array([0.64705882, 0.
, 0.14901961, 1.        ]), 862: array([0.64705882, 0.        , 0.14901961, 1.
]), 857: array([0.64705882, 0.        , 0.14901961, 1.        ]), 858: array([0.6
4705882, 0.        , 0.14901961, 1.        ]), 894: array([0.64705882, 0.
, 0.14901961, 1.        ]), 895: array([0.64705882, 0.        , 0.14901961, 1.
]), 769: array([0.87212611, 0.24921184, 0.18093041, 1.        ]), 772: array([0.8
7212611, 0.24921184, 0.18093041, 1.        ]), 774: array([0.87212611, 0.2492118
4, 0.18093041, 1.        ]), 798: array([0.87212611, 0.24921184, 0.18093041, 1.
]), 800: array([0.87212611, 0.24921184, 0.18093041, 1.        ]), 803: array([0.8
7212611, 0.24921184, 0.18093041, 1.        ]), 804: array([0.87212611, 0.2492118
4, 0.18093041, 1.        ]), 805: array([0.87212611, 0.24921184, 0.18093041, 1.
]), 810: array([0.87212611, 0.24921184, 0.18093041, 1.        ]), 811: array([0.8
7212611, 0.24921184, 0.18093041, 1.        ]), 819: array([0.87212611, 0.2492118
4, 0.18093041, 1.        ]), 823: array([0.87212611, 0.24921184, 0.18093041, 1.
]), 697: array([0.87212611, 0.24921184, 0.18093041, 1.        ]), 828: array([0.8
7212611, 0.24921184, 0.18093041, 1.        ]), 830: array([0.87212611, 0.2492118
4, 0.18093041, 1.        ]), 703: array([0.87212611, 0.24921184, 0.18093041, 1.
]), 708: array([0.87212611, 0.24921184, 0.18093041, 1.        ]), 840: array([0.8
7212611, 0.24921184, 0.18093041, 1.        ]), 713: array([0.87212611, 0.2492118
4, 0.18093041, 1.        ]), 719: array([0.87212611, 0.24921184, 0.18093041, 1.
]), 856: array([0.87212611, 0.24921184, 0.18093041, 1.        ]), 729: array([0.8
7212611, 0.24921184, 0.18093041, 1.        ]), 861: array([0.87212611, 0.2492118
4, 0.18093041, 1.        ]), 863: array([0.87212611, 0.24921184, 0.18093041, 1.
]), 869: array([0.87212611, 0.24921184, 0.18093041, 1.        ]), 745: array([0.8
7212611, 0.24921184, 0.18093041, 1.        ]), 747: array([0.87212611, 0.2492118
4, 0.18093041, 1.        ]), 876: array([0.87212611, 0.24921184, 0.18093041, 1.
]), 878: array([0.87212611, 0.24921184, 0.18093041, 1.        ]), 880: array([0.8
7212611, 0.24921184, 0.18093041, 1.        ]), 753: array([0.87212611, 0.2492118
4, 0.18093041, 1.        ]), 882: array([0.87212611, 0.24921184, 0.18093041, 1.
]), 886: array([0.87212611, 0.24921184, 0.18093041, 1.        ]), 888: array([0.8
7212611, 0.24921184, 0.18093041, 1.        ]), 889: array([0.87212611, 0.2492118
4, 0.18093041, 1.        ]), 890: array([0.87212611, 0.24921184, 0.18093041, 1.
]), 864: array([0.97485582, 0.557401  , 0.32272203, 1.        ]), 825: array([0.9
7485582, 0.557401  , 0.32272203, 1.        ]), 884: array([0.97485582, 0.557401
, 0.32272203, 1.        ]), 893: array([0.97485582, 0.557401  , 0.32272203, 1.
]), 867: array([0.99515571, 0.83229527, 0.52133795, 1.        ]), 872: array([0.9
9515571, 0.83229527, 0.52133795, 1.        ]), 873: array([0.99515571, 0.8322952
7, 0.52133795, 1.        ]), 776: array([0.99515571, 0.83229527, 0.52133795, 1.
]), 874: array([0.99515571, 0.83229527, 0.52133795, 1.        ]), 877: array([0.9
9515571, 0.83229527, 0.52133795, 1.        ]), 885: array([0.99515571, 0.8322952
7, 0.52133795, 1.        ]), 887: array([0.99515571, 0.83229527, 0.52133795, 1.
]), 859: array([0.99515571, 0.83229527, 0.52133795, 1.        ]), 860: array([0.9
976163 , 0.99907728, 0.75340254, 1.        ]), 875: array([0.82137639, 0.9249519
4, 0.95640138, 1.        ]), 883: array([0.55640138, 0.75963091, 0.8638985 , 1.
]), 891: array([0.31034218, 0.50618993, 0.73041138, 1.        ]), 892: array([0.1
9215686, 0.21176471, 0.58431373, 1.        ])}
```

698 Network with Girvan-Newman Communities (different colors)



698 Network with Girvan-Newman Communities, top communities blue, rest orange




```
Detected Communities: ({865, 866, 868, 870, 871, 879, 881, 862, 857, 858, 894, 89
5}, {769, 772, 774, 798, 800, 803, 804, 805, 810, 811, 819, 823, 697, 828, 830, 7
03, 708, 840, 713, 719, 856, 729, 861, 863, 869, 745, 747, 876, 878, 880, 753, 88
2, 886, 888, 889, 890}, {864, 825, 884, 893}, {867, 872, 873, 776, 874, 877, 885,
887, 859}, {860}, {875}, {883}, {891}, {892})
```

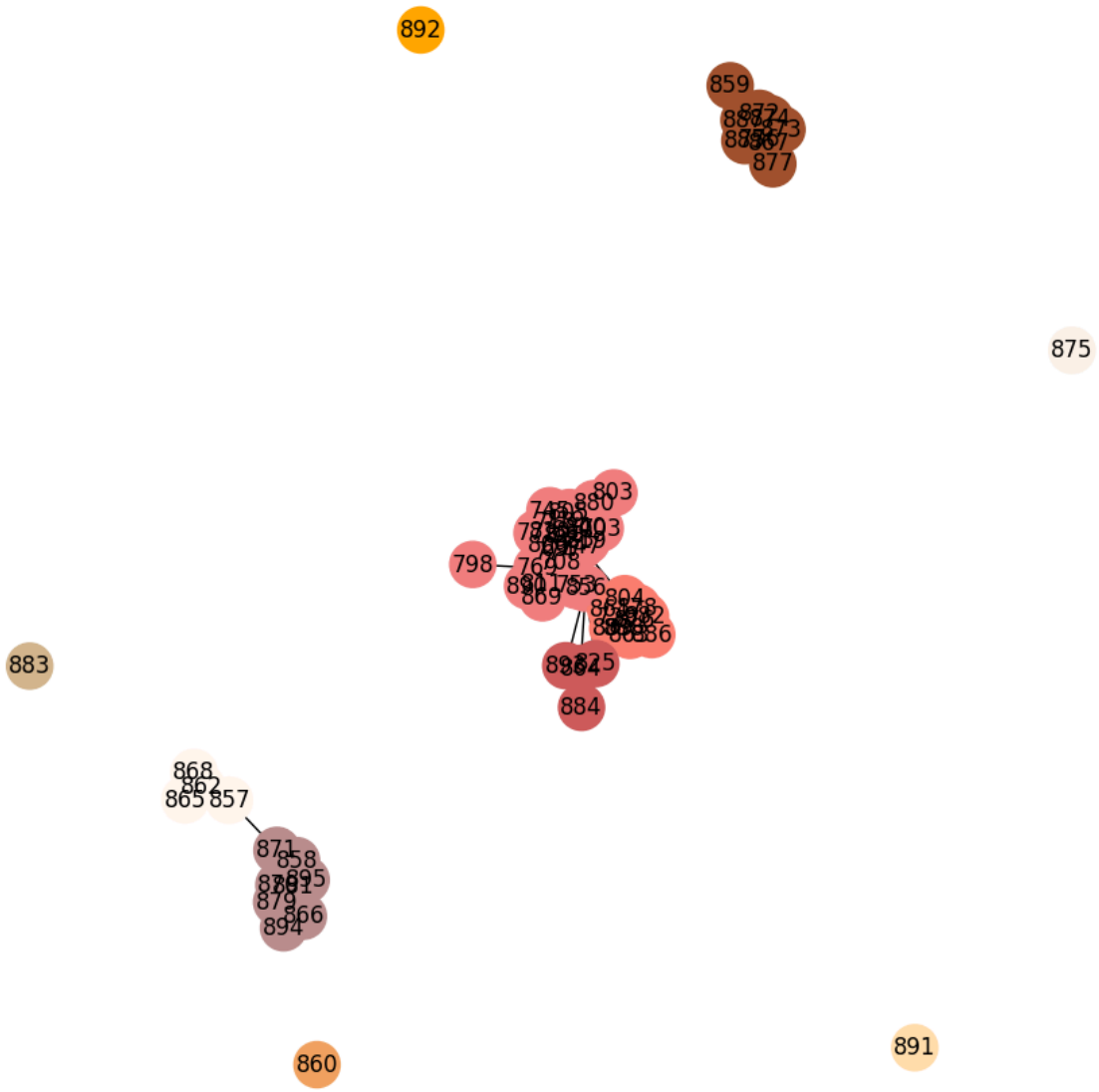
Task 4.2.3 (4 p)

A graph can be split into clusters by the connectivity, contents or with both connectivity and contents of the graph. In social network graphs individuals may belong to many groups or communities so the clustering is not strictly defined for this kind of data. A graph can be partitioned in many ways and the graph partition problem is NP hard to find the best partition. A big computation cluster seems to be the only way to find good solution for large graphs quickly. As seen in the course book chapter Mining Social-Network Graphs, there are many different graph clustering/partitioning/grouping algorithms available. In the following code, Label Propagation Algorithm is used to cluster communities. The code uses the data files from the last example.

1. Run the code. Examine the resulting graph. How many clusters the LPA algorihtm generated?

4 main clusters, 5 outliers.

In [18]:
```
#%matplotlib notebook
#!pip install igraph
#!pip install graphframes
import networkx as nx
import matplotlib.pyplot as plt
import numpy as np
#import igraph as ig

#readn graph edges(or arcs, lines) (multiple edges per vertex can exist)
#make pyspark dataframe form the data
lines = sc.textFile("698.edges")
edges = lines.map(lambda l: l.split(" ")). \
            map(lambda p: Row( src=int(p[0]), dst=int(p[1])) )
edges = sqlContext.createDataFrame(edges)

#read graph vertices(or nodes, points) (these are unique)
lines = sc.textFile("698.feat")
vertices = lines.map(lambda l: l.split(" ")). \
            map(lambda p: Row(id=int(p[0]), name="userid_"+p[0] ,features=[int(x
vertices = sqlContext.createDataFrame(vertices)

#Add nodes, vertices, weights and labels to graph
G = nx.Graph()
for x in edges.collect():
    G.add_edges_from([(x[0],x[1])], weight=1)
for x in vertices.collect():
    G.add_node(x[0],label=x[1])


#Label probagation algorithm (LPA)
lst_b=nx.community.label_propagation_communities(G)
#lst_b = nx.community.greedy_modularity_communities(G)

#Plot image
#colors = ["#00C98D","#00C90E", "#5030C0", "#5030D0", "#5040D0", "#50F0F0"]
#reserve more colors than needed, https://stackoverflow.com/questions/22408237/n
colors=['rosybrown','lightcoral','indianred','salmon','sienna','seashell','sandy
pos = nx.spring_layout(G)
color_map_b = {}
keys = G.nodes()
values = "black"
for i in keys:
        color_map_b[i] = values
counter = 0
for x in lst_b:
  for n in x:
    color_map_b[n] = colors[counter]
  counter = counter + 1
plt.figure(figsize=(11, 11))
nx.draw_networkx_edges(G, pos)
nx.draw_networkx_nodes(G, pos, node_color=dict(color_map_b).values(),node_size=6
nx.draw_networkx_labels(G, pos)
plt.title("698 Network communities with selected individual colors, random locat
plt.axis("off")
plt.show()

##Alternative
```

```
##plotting, generate unique colors for each group -----------------------

#grouplabels = [list(x[1].values())[0] for x in G.nodes(True)]
#node_texts = {node:node for node in G.nodes()};
#cmap = plt.get_cmap('gist_rainbow')
#uniqlabels = np.unique(grouplabels)
#randvals = np.random.random_sample((len(uniqlabels),1))
#colorlut = dict(zip(uniqlabels,randvals))
#gcolors = []
#for x in grouplabels:
#    gcolors.append(cmap(float(colorlut[x])))
#positions=nx.spring_layout(G,k=0.1,scale=1.5,iterations=20)
#plt.figure(figsize=(11, 11))
#nx.draw_networkx(G,positions, labels=node_texts, node_color = gcolors, node_siz
#plt.title("698 Network communities with rainbow colors, notice random locations
#plt.axis("off")
#plt.show()
##plotting end ------------------------------------------------------------
```

698 Network communities with selected individual colors, random locations